

CS3002E Compiler Design

Monsoon 2025

Lecture #27

Translation of While loop, Sequence

Department of Computer Science & Engineering
NIT Calicut

Translation of *while* loop

Example: 3-address code generated for
while ($i < 10$) $i = i + 1$:

L_1 : if $i < 10$ goto L_2

goto L_3

L_2 : $t_1 = i + 1$

$i = t_1$

goto L_1

L_3 :

Translation of *while* loop

Production	Semantic Rules
$S \rightarrow \text{while } (B) S_1$	$\begin{aligned} &begin = \text{newLabel}() \\ &B.true = \text{newLabel}() \\ &B.false = S.next \\ &S_1.next = begin \\ &S.code = \text{label}(begin) \parallel B.code \parallel \text{label}(B.true) \\ &\parallel S_1.code \parallel \text{gen}('goto' \text{ } begin) \end{aligned}$

- *begin* is a local variable that holds the label attached to the first instruction of *B*.
- *B.true* is attached to the first instruction of *S*₁.
- The last instruction *goto begin* is to transfer control to the beginning of *B.code* after each iteration of *S*₁.

Exercise: Why is *S*₁.*next* set as *begin* ?

Translation of Sequence of Statements

Production	Semantic Rules
$S \rightarrow S_1 S_2$	$S_1.next = newLabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Translation of Sequence of Statements

Production	Semantic Rules
$S \rightarrow S_1 S_2$	$S_1.next = newLabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Exercise: For each of the following sequence of statements, draw a parse tree and show how the inherited attribute $S.next$ is set for each individual statement and statement sequence. Write the generated 3-address code.

1. $x = a \quad y = b$
2. $x = a \quad y = b \quad z = c$

Translation of Program

Production	Semantic Rules
$P \rightarrow S$	$S.next = newLabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow S_1 S_2$	$S_1.next = newLabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

Exercise

Write the 3-address code generated for the following instruction, as per the given SDD:

```
while (i < 10 && sum < 100)  
    sum = sum + i  
    i = i + 1
```

Filling up target labels

- The attributes *B.true*, *B.false* and *S.next* are inherited.
- Generated code will contain jump instructions with target labels unspecified.
- A second pass will be required for binding labels to addresses.

Backpatching: Code generation in one pass

For nonterminal B , use synthesized attributes:

- $B.truelist$ to maintain the list of instructions into which the value of $B.true$ is to be inserted.
- $B.falselist$ to maintain the list of instructions into which the value of $B.false$ is to be inserted.

For nonterminal S , use synthesized attribute

- $S.nextlist$ to maintain the list of instructions into which the value of $S.next$ is to be inserted.

When the value of a label becomes known, the target labels in the instructions in the associated list are updated with their actual value.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Section 6.6
- Interested students may read Section 6.7 for details on backpatching.